

Proyecto 3: Intro a la IA

Salomón Avila, Gabriel Jaramillo y Ana Romero

Mayo 2026

Índice

1. Introducción	3
2. ¿Qué es una Red Bayesiana?	3
3. Arquitectura General del Proyecto	3
4. Clase VariableAleatoria	4
4.1. Representación de la CPT	4
5. Clase RedBayesiana	4
6. Construcción del Grafo	5
7. Carga de Variables	5
8. Carga de Probabilidades	5
9. Problema Encontrado: Dependencia del Orden	6
10.Solución: Normalización	6
11.Inferencia Exacta por Enumeración	6
12.Algoritmo ENUMERATE-ALL	7
13.Orden Topológico	7
14.Error Crítico Encontrado	7
15.Generalización de Variables	8
16.Parseo de Consultas	8
17.Normalización Final	8

18.Ejemplo de Inferencia	9
19.Manejo de Memoria	9
20.Conclusiones	9
21.Pruebas	10
21.1. Carga de Variables	10
21.2. Carga de Probabilidades	11
21.3. Tabla de Probabilidades	12
21.4. Consulta por Enumeración	13
21.5. Ejemplo 2: Red de Gripe	14

1. Introducción

El objetivo de este proyecto fue desarrollar un motor de inferencia basado en redes bayesianas utilizando C++. El sistema permite:

- Construir una red bayesiana desde archivos de texto.
- Representar variables aleatorias y sus relaciones de dependencia.
- Cargar tablas de probabilidad condicional (CPTs).
- Consultar probabilidades locales.
- Realizar inferencia exacta mediante el algoritmo de enumeración.

El proyecto fue desarrollado usando programación modular y estructuras abstractas de datos (TADs).

2. ¿Qué es una Red Bayesiana?

Una red bayesiana es un modelo probabilístico representado mediante un grafo dirigido acíclico (DAG).

En este modelo:

- Cada nodo representa una variable aleatoria.
- Cada arista representa una dependencia probabilística.
- Cada nodo tiene asociada una tabla de probabilidad condicional.

Ejemplo de red utilizada:

```
NUBLADO → LLUVIA
NUBLADO → ROCIADOR
LLUVIA → HIERBAHUMEDA
ROCIADOR → HIERBAHUMEDA
```

3. Arquitectura General del Proyecto

El proyecto se dividió en módulos independientes:

```
main.cpp
TADS/
  VariableAleatoria.h/.cpp
  RedBayesiana.h/.cpp
Utils/
  utils.h/.cpp
```

Cada archivo tiene una responsabilidad específica.

4. Clase VariableAleatoria

La clase `VariableAleatoria` representa un nodo de la red bayesiana. Cada variable almacena:

- Nombre de la variable.
- Lista de padres.
- Lista de hijos.
- Valores posibles.
- Tabla de probabilidad condicional.

4.1. Representación de la CPT

La CPT se almacenó usando:

```
1 map<string, map<string, double>> tablaProbabilidadCondicional;
```

Esta estructura permite almacenar:

condicion -> valor -> probabilidad

Ejemplo:

"LLUVIA=true,ROCIADOR=false"

↓

"true" -> 0.90

"false" -> 0.10

5. Clase RedBayesiana

La clase `RedBayesiana` administra toda la red probabilística. Entre sus responsabilidades están:

- Crear variables.
- Conectar nodos.
- Cargar probabilidades.
- Ejecutar inferencia.
- Obtener orden topológico.

6. Construcción del Grafo

Las conexiones se crean usando:

```
1 void RedBayesiana::agregarConexiones(string padre, string hijo)
```

Internamente:

```
1 variablePadre->agregarHijo(variableHijo);  
2 variableHijo->agregarPadre(variablePadre);
```

7. Carga de Variables

El archivo de variables tiene el formato:

```
NUBLADO LLUVIA  
NUBLADO ROCIADOR  
LLUVIA HIERBAHUMEDA  
ROCIADOR HIERBAHUMEDA
```

Cada línea representa:

```
PADRE HIJO
```

8. Carga de Probabilidades

Las probabilidades se cargan desde un archivo de texto:

```
NUBLADO  
true 0.5  
false 0.5  
  
LLUVIA | NUBLADO  
NUBLADO=true true 0.8  
NUBLADO=false true 0.2
```

El sistema identifica:

- Variables sin padres.
- Variables con padres.
- Entradas de la CPT.

9. Problema Encontrado: Dependencia del Orden

Durante las pruebas apareció un problema importante.
Las siguientes condiciones:

`ROCIADOR=false,LLUVIA=true`

y

`LLUVIA=true,ROCIADOR=false`

representan exactamente lo mismo, pero el programa las interpretaba como claves distintas.

10. Solución: Normalización

La solución consistió en ordenar las condiciones alfabéticamente antes de almacenarlas y consultarlas.

```
1 sort(partes.begin(), partes.end());
```

Esto garantizó que:

`LLUVIA=true,ROCIADOR=false`

siempre tuviera el mismo formato.

11. Inferencia Exacta por Enumeración

La inferencia exacta se implementó mediante el algoritmo:

ENUMERATE-ALL

El objetivo es calcular:

$$P(X|e)$$

donde:

- X es la variable consulta.
- e es la evidencia observada.

12. Algoritmo ENUMERATE-ALL

La función principal fue:

```
1 enumerarTodas()
```

El algoritmo:

1. Recorre las variables en orden topológico.
2. Si una variable ya tiene valor:
 - usa directamente su probabilidad.
3. Si no:
 - prueba todos sus valores posibles.
 - suma los resultados.

13. Orden Topológico

La inferencia requiere que los padres sean evaluados antes que los hijos.
Para esto se implementó DFS:

```
1 dfsTopologico()
```

y luego:

```
1 reverse(orden.begin(), orden.end());
```

14. Error Crítico Encontrado

Durante las pruebas apareció el siguiente problema:

$P(\text{LLUVIA}=\text{true} \mid \text{evidencia}) = 1$

cuando claramente no debía valer 1.

El error estaba aquí:

```
1 suma += obtenerProbabilidad(varTem, true, evidencia)
```

La rama falsa también estaba usando `true`.

La solución correcta fue:

```
1 suma += obtenerProbabilidad(varTem, false, evidencia)
```

15. Generalización de Variables

Inicialmente el sistema asumía variables booleanas.
Luego se generalizó para soportar múltiples valores.
Antes:

```
1 map<string, bool>
```

Después:

```
1 map<string, string>
```

Esto permitió manejar estados como:

```
none  
light  
heavy
```

16. Parseo de Consultas

Las consultas tienen el formato:

```
P(LLUVIA=true | HIERBAHUMEDA=true)
```

La función:

```
1 parsearConsulta()
```

extrae:

- Variable consulta.
- Valor consultado.
- Evidencia.

17. Normalización Final

Las probabilidades calculadas inicialmente no suman 1.
Por esto se normalizan usando:

$$\alpha = \frac{1}{\sum P(X_i, e)}$$

Implementación:

```
1 par.second /= total;
```


18. Ejemplo de Inferencia

Consulta:

$P(\text{LLUVIA} \mid \text{HIERBAHUMEDA}=\text{true})$

Resultado aproximado:

$P(\text{LLUVIA}=\text{true} \mid \text{evidencia}) \quad 0.77$
 $P(\text{LLUVIA}=\text{false} \mid \text{evidencia}) \quad 0.23$

Interpretación:

Si la hierba está húmeda, es más probable que haya llovido.

19. Manejo de Memoria

Las variables fueron creadas dinámicamente usando:

```
1 new VariableAleatoria(nombre)
```

Por esto fue necesario liberar memoria en el destructor:

```
1 delete par.second;
```

20. Conclusiones

El proyecto permitió aplicar conceptos fundamentales de inteligencia artificial y programación:

- Redes Bayesianas.
- Inferencia probabilística.
- Ordenamiento topológico.
- Algoritmos recursivos.
- Programación modular.
- Manejo de memoria dinámica.

Además, el desarrollo permitió comprender cómo pequeños errores lógicos pueden alterar completamente los resultados probabilísticos y cómo la depuración paso a paso es esencial en sistemas de inferencia.

21. Pruebas

21.1. Carga de Variables

```
Ingrese el nombre del archivo donde estan las variables: variables
Variables cargadas.
Orden topologico:
RAIN MAINTENANCE TRAIN APPOINTMENT
```

(a) Carga de variables en consola

```
variables.txt X
variables.txt
1  RAIN MAINTENANCE
2  RAIN TRAIN
3  MAINTENANCE TRAIN
4  TRAIN APPOINTMENT
```

(b) Archivo variables.txt

Figura 1: Prueba de carga de variables

21.2. Carga de Probabilidades

```
Ingrese el nombre del archivo donde estan las probabilidades: probabilidades

Red creada.

Cargando probabilidades...

Datos cargados
Variable: APPOINTMENT
Padres: TRAIN

Variable: MAINTENANCE
Padres: RAIN

Variable: RAIN
Padres:

Variable: TRAIN
Padres: RAIN MAINTENANCE

Variable: APPOINTMENT
  TRAIN=delayed | attend -> 0.6
  TRAIN=delayed | miss -> 0.4
  TRAIN=on_time | attend -> 0.9
  TRAIN=on_time | miss -> 0.1

Variable: MAINTENANCE
  RAIN=heavy | no -> 0.9
  RAIN=heavy | yes -> 0.1
  RAIN=light | no -> 0.8
  RAIN=light | yes -> 0.2
  RAIN=none | no -> 0.6
  RAIN=none | yes -> 0.4
```

(a) Carga parcial

```
Variable: RAIN
(sin padres) | heavy -> 0.1
(sin padres) | light -> 0.2
(sin padres) | none -> 0.7

Variable: TRAIN
  MAINTENANCE=no,RAIN=heavy | delayed -> 0.5
  MAINTENANCE=no,RAIN=heavy | on_time -> 0.5
  MAINTENANCE=no,RAIN=light | delayed -> 0.3
  MAINTENANCE=no,RAIN=light | on_time -> 0.7
  MAINTENANCE=no,RAIN=none | delayed -> 0.1
  MAINTENANCE=no,RAIN=none | on_time -> 0.9
  MAINTENANCE=yes,RAIN=heavy | delayed -> 0.6
  MAINTENANCE=yes,RAIN=heavy | on_time -> 0.4
  MAINTENANCE=yes,RAIN=light | delayed -> 0.4
  MAINTENANCE=yes,RAIN=light | on_time -> 0.6
  MAINTENANCE=yes,RAIN=none | delayed -> 0.2
  MAINTENANCE=yes,RAIN=none | on_time -> 0.8
```

(b) Carga completa

```
≡ probabilidades.txt X
≡ probabilidades.txt
1  RAIN
2  none 0.7
3  light 0.2
4  heavy 0.1
5
6  MAINTENANCE | RAIN
7  RAIN=none yes 0.4
8  RAIN=none no 0.6
9  RAIN=light yes 0.2
10 RAIN=light no 0.8
11 RAIN=heavy yes 0.1
12 RAIN=heavy no 0.9
13
14 TRAIN | RAIN,MAINTENANCE
15 RAIN=none,MAINTENANCE=yes on_time 0.8
16 RAIN=none,MAINTENANCE=yes delayed 0.2
17 RAIN=none,MAINTENANCE=no on_time 0.9
18 RAIN=none,MAINTENANCE=no delayed 0.1
19 RAIN=light,MAINTENANCE=yes on_time 0.6
20 RAIN=light,MAINTENANCE=yes delayed 0.4
21 RAIN=light,MAINTENANCE=no on_time 0.7
22 RAIN=light,MAINTENANCE=no delayed 0.3
23 RAIN=heavy,MAINTENANCE=yes on_time 0.4
24 RAIN=heavy,MAINTENANCE=yes delayed 0.6
25 RAIN=heavy,MAINTENANCE=no on_time 0.5
26 RAIN=heavy,MAINTENANCE=no delayed 0.5
27
```

21.3. Tabla de Probabilidades

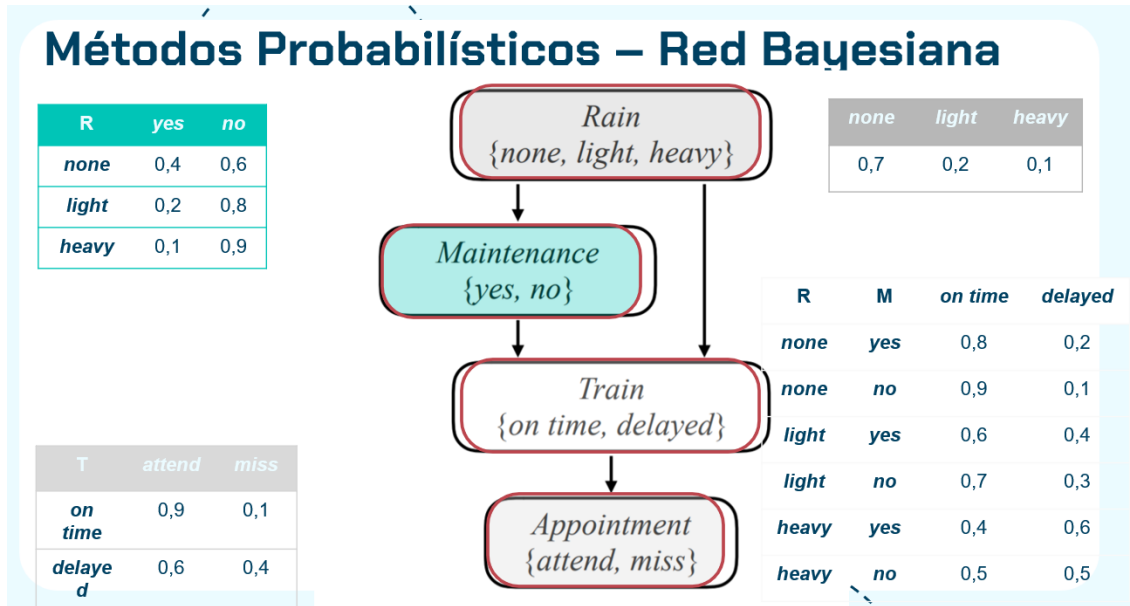


Figura 3: Tabla de probabilidad condicional generada

21.4. Consulta por Enumeración

```

Ingrese la consulta [formato: P(VARIABLE | EVIDENCIA=valor,EVIDENCIA=valor, ...): P(APPOINTMENT | RAIN=light,MAINTENANCE=no)

===== TRAZA DE INFERENCIA =====
Consulta : APPOINTMENT
Evidencia : MAINTENANCE=no RAIN=light
Orden topológico: RAIN MAINTENANCE TRAIN APPOINTMENT

-- Enumeración APPOINTMENT = attend --
[E] RAIN=light | p=0.2
[E] MAINTENANCE=no | p=0.8
[M] TRAIN (marginalizando)
  rain on time | p=0.7
  [E] APPOINTMENT=attend | p=0.9
  -> prob. parcial = 1.0
  rain delayed | p=0.3
  [E] APPOINTMENT=attend | p=0.6
  -> prob. parcial = 1.0
  suma marginalizada = 0.81
  Resultado sin normalizar: 0.1296

-- Enumeración APPOINTMENT = miss --
[E] RAIN=light | p=0.2
[E] MAINTENANCE=no | p=0.8
[M] TRAIN (marginalizando)
  rain on time | p=0.7
  [E] APPOINTMENT=miss | p=0.1
  -> prob. parcial = 1.0
  rain delayed | p=0.3
  [E] APPOINTMENT=miss | p=0.4
  -> prob. parcial = 1.0
  suma marginalizada = 0.19
  Resultado sin normalizar: 0.0304

-- Normalización (alfa = 1/0.16 = 6.25) --
P(APPOINTMENT=attend | evidencia) = 0.81
P(APPOINTMENT=miss | evidencia) = 0.19

```

(a) Consulta ingresada

```

P(APPOINTMENT=attend | evidencia) = 0.81
P(APPOINTMENT=miss | evidencia) = 0.19

```

(b) Resultado obtenido

Métodos Probabilísticos – Red Bayesiana

Inferencia Bayesiana por Enumeración – Ejemplo

Consulta: $P(\text{Appointment} | \text{Rain} = \text{light} \wedge \text{Maintenance} = \text{no})$

$P(\text{App} | \text{light} \wedge \text{no}) = \alpha P(\text{App} | \text{light} \wedge \text{no}) = \alpha \sum_{\text{Train}} [P(\text{App} | \text{light} \wedge \text{no} \wedge \text{Train})]$

Para Appointment = attend:

$$P(\text{attend} | \text{light} \wedge \text{no}) = \alpha \sum_{\text{Train}} [P(\text{light})P(\text{no} | \text{light})P(\text{t} | \text{light} \wedge \text{no})P(\text{attend} | \text{Train})]$$

$$= \alpha [P(\text{light})P(\text{no} | \text{light})P(\text{on time} | \text{light} \wedge \text{no})P(\text{attend} | \text{on time}) + P(\text{light})P(\text{no} | \text{light})P(\text{delayed} | \text{light} \wedge \text{no})P(\text{attend} | \text{delayed})]$$

$$= \alpha [0.2 \cdot 0.8 \cdot 0.7 \cdot 0.9 + 0.2 \cdot 0.8 \cdot 0.3 \cdot 0.6]$$

$$= \alpha [0.1008 + 0.0288]$$

$$= \alpha [0.1296]$$

(c) Comprobación manual 1

Métodos Probabilísticos – Red Bayesiana

Inferencia Bayesiana por Enumeración – Ejemplo

Consulta: $P(\text{Appointment} | \text{Rain} = \text{light} \wedge \text{Maintenance} = \text{no})$

$P(\text{App} | \text{light} \wedge \text{no}) = \alpha P(\text{App} | \text{light} \wedge \text{no}) = \alpha \sum_{\text{Train}} [P(\text{App} | \text{light} \wedge \text{no} \wedge \text{Train})]$

Para Appointment = miss:

$$P(\text{miss} | \text{light} \wedge \text{no}) = \alpha \sum_{\text{Train}} [P(\text{light})P(\text{no} | \text{light})P(\text{t} | \text{light} \wedge \text{no})P(\text{miss} | \text{Train})]$$

$$= \alpha [P(\text{light})P(\text{no} | \text{light})P(\text{on time} | \text{light} \wedge \text{no})P(\text{miss} | \text{on time}) + P(\text{light})P(\text{no} | \text{light})P(\text{delayed} | \text{light} \wedge \text{no})P(\text{miss} | \text{delayed})]$$

$$= \alpha [0.2 \cdot 0.8 \cdot 0.7 \cdot 0.1 + 0.2 \cdot 0.8 \cdot 0.3 \cdot 0.4]$$

$$= \alpha [0.0112 + 0.0192]$$

$$= \alpha [0.0304]$$

(d) Comprobación manual 2

Figura 4: Prueba de inferencia por enumeración

21.5. Ejemplo 2: Red de Gripe

```
gabri@gabriel:/mnt/c/Users/gabri/Desktop/Gabriel/Gabriel Universidad/se
mestre 6/Intro a la IA/proyecto3/Proyecto_3_Intro_a_la_IA$ ./programa
<-----> Bienvenido al motor de inferencia <----->

Menu:
1. Cargar variables
2. Cargar probabilidades
3. Obtener probabilidad local
4. Consulta por enumeracion
0. Salir
Ingrese la opcion: 1

Ingrese el nombre del archivo donde estan las variables: variables2

Variables cargadas.

Orden topologico:
FEVER FLU FATIGUE COLD HEADACHE
```

(a) Carga de variables

```
Datos cargados
Variables: COLD
Padres: FEVER

Variable: FATIGUE
Padres: FLU

Variable: FEVER
Padres:

Variable: FLU
Padres: FEVER

Variable: HEADACHE
Padres: FLU COLD

Variable: COLD
FEVER=high | no -> 0.8
FEVER=high | yes -> 0.2
FEVER=low | no -> 0.7
FEVER=low | yes -> 0.3

Variable: FATIGUE
FLU=no | no -> 0.8
FLU=no | yes -> 0.2
FLU=yes | no -> 0.2
FLU=yes | yes -> 0.8

Variable: FEVER
(sin padres) | high -> 0.4
(sin padres) | low -> 0.6

Variable: FLU
FEVER=high | no -> 0.3
FEVER=high | yes -> 0.7
FEVER=low | no -> 0.9
FEVER=low | yes -> 0.1
```

(b) Carga de probabilidades

```
Ingrese la consulta [Formato: P(VARIABLE | EVIDENCIA1=valor,EVIDENCIA2=valor, ...)] : P(FATIGUE | FEVER=high)
```

(c) Consulta

$P(\text{FATIGUE}=\text{no} \mid \text{evidencia}) = 0.38$
 $P(\text{FATIGUE}=\text{yes} \mid \text{evidencia}) = 0.62$

(d) Resultado

Consulta: $P(\text{FATIGUE} \mid \text{FEVER}=\text{high})$

$P(\text{FEVER}=\text{high}) = 0.4$

$P(\text{FLU}=\text{yes} \mid \text{FEVER}=\text{high}) = 0.7$

$P(\text{FLU}=\text{no} \mid \text{FEVER}=\text{high}) = 0.3$

(e) Verificación 1

Para $\text{FATIGUE} = \text{yes}$

$P(\text{Fatigue}=\text{yes}, \text{Flu}=\text{yes} \mid \text{Fever}=\text{high}) = P(\text{FLU}=\text{yes} \mid \text{FEVER}=\text{high}) \times P(\text{Fatigue}=\text{yes} \mid \text{Flu}=\text{yes})$
 $= 0.7 \times 0.8 = 0.56$

$P(\text{Fatigue}=\text{yes}, \text{Flu}=\text{no} \mid \text{Fever}=\text{high}) = P(\text{FLU}=\text{no} \mid \text{FEVER}=\text{high}) \times P(\text{Fatigue}=\text{yes} \mid \text{Flu}=\text{no})$
 $= 0.3 \times 0.2 = 0.06$

$P(\text{FATIGUE}=\text{yes} \mid \text{FEVER}=\text{high}) = 0.56 + 0.06 = 0.62$

Para $\text{FATIGUE} = \text{no}$

$P(\text{Fatigue}=\text{no}, \text{Flu}=\text{yes} \mid \text{Fever}=\text{high}) = P(\text{FLU}=\text{yes} \mid \text{FEVER}=\text{high}) \times P(\text{Fatigue}=\text{no} \mid \text{Flu}=\text{yes})$
 $= 0.7 \times 0.2 = 0.14$

$P(\text{Fatigue}=\text{no}, \text{Flu}=\text{no} \mid \text{Fever}=\text{high}) = P(\text{FLU}=\text{no} \mid \text{FEVER}=\text{high}) \times P(\text{Fatigue}=\text{no} \mid \text{Flu}=\text{no})$
 $= 0.3 \times 0.8 = 0.24$

$P(\text{FATIGUE}=\text{no} \mid \text{FEVER}=\text{high}) = 0.14 + 0.24 = 0.38$

(f) Verificación 2

Figura 5: Pruebas usando la red bayesiana de gripa